

RELATÓRIO – API REST INSTRUMENTOS MUSICAIS

Daniela Silva de Jesus | RA: 12724211599

Larissa Aparecida Bairon da Silva Sena | RA: 12725234713

Victor Elisio dos Santos Silva | RA: 1272426873

Descrição do Projeto

O projeto consiste em uma API REST desenvolvida em JavaScript utilizando Node.js e Express para gerenciamento de uma loja virtual de instrumentos musicais.

A aplicação implementa funcionalidades de:

- Gerenciamento de clientes;
- Gerenciamento de vendedores;
- Gerenciamento de produtos;
- Gerenciamento de pedidos;
- Controle de estoque;
- Processamento de pagamentos;
- Controle de carrinho de compras;
- Geração de relatórios estatísticos;
- Controle de autenticação e autorização via JWT.

O sistema utiliza banco de dados PostgreSQL e é executado através de containers Docker.

Requisitos de Software

Para execução da aplicação são necessários:

Software

- Docker Desktop
- Docker Compose
- Git
- Navegador Web

Backend

- Node.js
- Express.js

Banco de Dados

- PostgreSQL

Documentação

- Swagger OpenAPI

Segurança

- JSON Web Token (JWT)

Containerização

- Docker
- Docker Compose

Controllers

Responsáveis por receber as requisições HTTP e retornar as respostas.

DAO (Data Access Object)

Responsáveis pelo acesso ao banco de dados.

Routes

Responsáveis pelo mapeamento das rotas da API.

Middleware

Responsável pela autenticação e autorização dos usuários.

Config

Responsável pelas configurações do banco de dados, JWT e Swagger.

Arquitetura Utilizada

A aplicação utiliza arquitetura em camadas (Layered Architecture):

Cliente → Rotas → Controllers → DAO → Banco de Dados

Benefícios:

- Separação de responsabilidades;
- Facilidade de manutenção;
- Reutilização de código;
- Escalabilidade.

Padrões de Projeto Utilizados

DAO (Data Access Object)

Separa o acesso ao banco de dados da lógica de negócio.

MVC (Model View Controller)

Organização do sistema em:

- Rotas
- Controllers
- Persistência de dados

Middleware Pattern

Utilizado para autenticação JWT.

Singleton

Aplicado na conexão com o banco PostgreSQL.

Instalação da Aplicação

Passo 1 – Clonar o Repositório

```
git clone
```

Passo 2 – Entrar na Pasta do Projeto

```
cd API-Rest-Instrumentos-Musicais
```

Passo 3 – Configurar Variáveis de Ambiente

Criar arquivo .env baseado no .env.example.

Exemplo:

PORT=3000

JWT_SECRET=senhaSegura

DATABASE_URL=postgres://usuario@postgres:5432/instrumentos

Passo 4 – Executar Docker

docker compose up –build

Passo 5 – Verificar Containers

docker ps

Inicialização da API

Após a execução dos containers, acessar:

<http://localhost:3000>

Documentação Swagger:

<http://localhost:3000/api-docs>

Credenciais de Acesso

Senha Única para TODOS os usuários: **senha123**

Login Administrador: admin@sistema.com

Login Vendedor (Exemplo): carlos@vendas.com

Login Cliente (Exemplo): joao@cliente.com

Fluxo de Utilização da API

Login

Realizar autenticação utilizando:

POST /auth/login

Exemplo:

```
{  
  "email": "admin@sistema.com",
```

```
“senha”: “senha123”  
}
```

A API retornará um token JWT.

Utilização do Token

Adicionar o token no cabeçalho:

Authorization: Bearer TOKEN

Cadastro de Produtos

POST /produtos

Permite cadastrar novos instrumentos musicais.

Consulta de Produtos

GET /produtos

Retorna todos os produtos cadastrados.

Cadastro de Clientes

POST /clientes

Permite cadastrar novos clientes.

Cadastro de Vendedores

POST /vendedores

Permite cadastrar vendedores.

Realização de Pedidos

POST /pedidos

Permite registrar uma venda.

Relatórios Estatísticos

A aplicação possui um serviço dedicado para geração de relatórios.

Relatórios disponíveis:

Produtos Mais Vendidos

Apresenta ranking dos produtos com maior volume de vendas.

Produto por Cliente

Apresenta quais produtos foram adquiridos por cada cliente.

Consumo Médio do Cliente

Apresenta o valor médio gasto por cliente.

Produtos com Baixo Estoque

Apresenta produtos que necessitam reposição.

Decisões Técnicas

As principais decisões adotadas pela equipe foram:

- Utilização de API REST por facilitar integração entre sistemas.
- Utilização do PostgreSQL devido à confiabilidade e desempenho.
- Utilização do Docker para padronização do ambiente.
- Utilização do JWT para segurança das rotas.
- Separação da geração de relatórios em serviço próprio para atender ao requisito da disciplina.

Conclusão

O sistema desenvolvido atende aos requisitos propostos pela disciplina de Sistemas Distribuídos e Mobile, utilizando tecnologias modernas de desenvolvimento backend, banco de dados relacional, autenticação segura e execução em containers Docker.

A arquitetura escolhida proporciona organização, escalabilidade e facilidade de manutenção, permitindo futuras expansões da aplicação.